Lab Session 4: Network Packet Capture & Traffic Analysis

Date: 8 July 2026

Duration: 1 hour

Scenario:

Monitoring network traffic at a hospital IT department. A user's machine is behaving strangely - sending unusual outbound traffic at odd hours. The task is to capture and analyse packets to determine if a data exfiltration or C2 (Command and Control) connection is occurring. Metasploitable 2 served as the target machine, with FTP and HTTP traffic deliberately generated to simulate real network activity to analyse.

Objective:

Capture live network traffic using tcpdump from the terminal, save it to a PCAP file, and analyse it in Wireshark. The goal was to identify protocols, extract credentials from unencrypted traffic, and understand how to spot suspicious patterns in a network capture - skills directly applicable to incident response and network forensics.

Environment:

Virtual Machines: Kali Linux (Live USB) +(Metasploitable 2 (VB)

Captures: capture.pcap (FTP traffic) + http_capture.pcap (HTTP traffic)

Tools Used: tcpdump (CLI) + Wireshark 4.6.4 (GUI)

Target: 192.168.56.105 (Metasploitable 2)

Pre LAB isssues

There were no issues with startup this time, Metasploitable launched cleanly

Firs Attempt, Capturing Packets with tcpdump:

Before capturing,I needed to identify the correct network interface connecting Kali to Metasploitable :

ip a

sudo tcpdump -D

The *ip a* output showed four interfaces: lo (loopback), eth0 (DOWN - no cable), wlan0 (WiFi, 192.168.8.12), and vboxnet0 (UP, 192.168.56.1 - the VirtualBox host-only network). The tcpdump -D output confirmed vboxnet0 as interface 2, listed as 'Up, Running, Connected'.

Next

I ran a live capture on vboxnet0 in verbose mode to observe traffic in real time this was done using:

sudo tcpdump -i vboxnet0 -v

Packets immediately appeared, Metasploitable was already generating background traffic including NetBIOS/UDP broadcasts. I then used ctrl+c after observing the live feed that confirmed the capture was working correctly.

After that I Captured and Saved to PCAP File:

To save the capture for Wireshark analysis, I started a background tcpdump saving to a PCAP file, then I generated deliberate FTP traffic in a second termkinal to create meaningful data to analyse.

Terminal 1 - capture running:

sudo tcpdump -i vboxnet0 -w capture.pcap

Terminal 2 - generating FTP traffic:

ftp 192.168.56.105

 # Username: msfadmin

 # Password: msfadmin

 # Command: ls

 # Command: exit

After the FTP session completed, I pressed Ctrl+C to stop the capture. tcpdump reported 79 packets captured, 0 dropped.

Searching for Credentials in the Capture

Before opening Wireshark, I searched directly for the PCAP file from the terminal to demonstrate that credentials are visible in raw packet data:

sudo tcpdump -r capture.pcap -A | grep -i 'password\|user\|msfadmin'

The output showed the full FTP login sequence in plaintext — the username and password were clearly visible in the packet payload.

Analysing Traffic in Wireshark

Loading the Capture

sudo wireshark &

The & ran Wireshark in the background so the terminal remained usable. Wireshark 4.6.4 launched and displayed the interface selection screen, showing

wlan0, vboxnet0, any, lo, eth0 and bluetooth interfaces. I opened the capture file via File → Open → capture.pcap

Applying Display Filters

Wireshark display filters allow isolation of specific traffic types. I applied an FTP filter to show only FTP packets from the 79 captured:

ftp

This immediately filtered the view to show only FTP protocol packets, making it easy to identify the authentication sequence without scrolling through ARP, TCP, and other background traffic.

Following the TCP Stream

The most powerful feature for credential extraction is TCP Stream reconstruction. Right-clicking an FTP packet and selecting Follow → TCP Stream reconstructed the entire FTP conversation in order, displaying it as human-readable text:

220 (vsFTPd 2.3.4)

USER msfadmin

331 Please specify the password.

PASS msfadmin

230 Login successful.

SYST

215 UNIX Type: L8

The stream showed the complete FTP session from connection banner through authentication to the directory listing command. The username and password are displayed in full.

HTTP Traffic Analysis

I then generated a separate HTTP capture to demonstrate web traffic analysis:

sudo tcpdump -i vboxnet0 -w http_capture.pcap

curl http://192.168.56.105

Opening http_capture.pcap in Wireshark and applying the http filter showed the HTTP GET request. Following the TCP stream revealed the full HTTP request headers in plaintext, including the GET method, Host, User-Agent (curl), and Accept headers — demonstrating that HTTP web traffic is equally vulnerable to packet capture analysis.

File → Export Objects → HTTP was also attempted to extract transferred files, but the curl request returned a minimal response with no downloadable objects. In a real scenario with a full web browsing session, this feature would extract images, scripts, and documents transferred over HTTP.

Statistics Analysis

Wireshark's Statistics menu provided three key views for traffic analysis:

- Protocol Hierarchy (Statistics → Protocol Hierarchy): showed the breakdown of all protocols in http_capture.pcap — 100% Ethernet/IPv4/TCP with 20% HTTP (1 of 5 packets). In a real investigation this view quickly reveals unexpected protocols that shouldn't be present on a network.

- Conversations (Statistics → Conversations): showed all host pairs that communicated during the capture, with packet counts and byte totals per conversation — useful for identifying which hosts are talking the most or communicating with unknown external IPs.
- IO Graphs (Statistics → IO Graphs): visualised traffic volume over time as a graph. Spikes during off-hours (e.g. 3AM) would be a red flag for scheduled data exfiltration or botnet activity.

Findings:

- FTP transmits both usernames and passwords in plaintext — visible in raw tcpdump output and clearly readable in Wireshark TCP Stream reconstruction.
- HTTP traffic is equally unprotected — request headers, URLs, and response content are all visible to anyone capturing network traffic.
- tcpdump's grep functionality allows rapid credential searching without opening a GUI — critical for quick triage on compromised systems.
- Wireshark's TCP Stream follow feature reconstructs entire conversations, making it trivial to read FTP sessions, HTTP requests, and Telnet commands.
- The vboxnet0 interface carries all VM-to-VM traffic in a host-only VirtualBox setup — the correct interface to monitor when analysing communications between Kali and Metasploitable.
- Protocol Hierarchy, Conversations, and IO Graphs are the three key statistical views for anomaly detection in network forensics.

Lessons Learned

- Encryption is not optional — any service transmitting credentials or sensitive data over an unencrypted protocol is vulnerable to passive interception, no active attack required.
- Always confirm the correct capture interface first (ip a + tcpdump -D) — capturing on the wrong interface wastes the entire exercise.
- Generating deliberate traffic in a second terminal while capturing in the first is the most reliable way to produce meaningful data to analyse.
- A quick tcpdump | grep pass is often faster than opening Wireshark for a first-pass credential check.

Note to Self:

Always confirm the interface with ip a and tcpdump -D before starting a capture, and generate deliberate traffic in a second terminal so there's something meaningful to analyse. A quick tcpdump | grep is a fast first-pass credential check before switching to Wireshark for deeper analysis.

What's Next?

Lab 05 — Exploitation and Post Exploitation Basics